DEDICATION

To Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam.

To my dear, First name.

To Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi.

To my dear, First name.

To .....

To **all of you**,

I dedicate this work.

First name NAME

Acknowledgement

TABLE OF CONTENTS

LIST OF FIGURES **vi**

LIST OF TABLES **vii**

LIST OF ABBREVIATIONS **viii**

GENERAL INTRODUCTION **1**

1 Contexte Industriel et Enjeux de la Sécurité Cloud **2**

1.1 Introduction 3

1.2 Organisation d'accueil 3

1.3 Le paradigme de l'infrastructure programmable 3

1.3.1 Convergence du Cloud Computing et de la culture DevOps 3

1.3.2 Infrastructure as Code : l'infrastructure devenue logiciel 4

1.4 Les nouveaux vecteurs d'insécurité dans les environnements Cloud 4

1.4.1 Fuite de secrets et compromission d'identités 4

1.4.2 Misconfigurations d'infrastructure as Code 5

1.4.3 Configuration Drift et perte de contrôle opérationnel 6

1.5 Problématique : le trilemme de la sécurité Cloud 7

1.5.1 Le conflit des priorités : Vélocité vs Sécurité vs Gouvernance 7

1.5.2 Synthèse des défis et problématique 8

1.6 Étude comparative et positionnement stratégique 8

1.6.1 Analyse des approches actuelles 9

1.6.2 Tableau comparatif détaillé 9

1.7 Présentation du projet 9

1.7.1 Vision du projet 10

1.7.2 Objectifs spécifiques 10

1.8 Méthodologie de travail 10

1.8.1 Présentation du processus 2TUP 11

1.8.2 Justification du choix pour un projet DevSecOps 12

1.9 Conclusion 12

2 État de l'Art : Sécurité Cloud et Technologies DevSecOps 13

2.1 Introduction 15

2.2 Sécurité du Cloud : modèle de responsabilité et Zero Trust 15

2.2.1 Le modèle de responsabilité partagée 15

2.2.2 Zero Trust : principe structurant de la gouvernance cloud 16

2.3 Évolution vers le DevSecOps 17

2.3.1 Du DevOps au DevSecOps 17

2.3.2 Shift-Left et Shift-Right Security 18

2.3.2.1 Shift-Left Security 19

2.3.2.2 Shift-Right Security 19

2.3.3 Métriques DORA appliquées à la sécurité 20

2.4 Policy-as-Code : Modélisation avancée de la décision 21

2.4.1 Définition et principes fondamentaux 21

2.4.2 Architecture PDP/PEP : Un système distribué critique 22

2.4.3 Open Policy Agent (OPA) et l'écosystème 24

2.4.4 Cas d'usage, avantages et limites 25

2.4.4.1 Cas d'usage en DevSecOps et cloud 25

2.4.4.2 Avantages 26

2.4.4.3 Limites 26

2.5 Écosystème d'outils DevSecOps : analyse comparative 27

2.5.1 Catégorisation fonctionnelle 27

2.5.2 Outils Shift-Left 28

2.5.2.1 Détection de secrets 28

2.5.2.2 Analyse de l'Infrastructure as Code 28

2.5.2.3 Analyse de vulnérabilités et de conteneurs 29

2.5.3 Moteurs de Policy-as-Code 29

2.5.4 Outils Shift-Right 30

2.5.4.1 Audit de posture cloud 30

2.5.4.2 Remédiation pilotée par politique 31

2.5.5 Plateformes transverses de gestion des findings 32

2.5.6 Synthèse comparative 32

2.6 Architecture d'orchestration des contrôles de sécurité 34

2.6.1 La crise de la normalisation et les contrats de données 34

2.6.2 Chaîne de traitement d'un pipeline DevSecOps 35

2.6.3 Enjeux de passage à l'échelle (Scalabilité et Performance) 36
2.7 Synthèse critique : lacunes dictant l'architecture cible 37
2.8 Conclusion 38

GENERAL CONCLUSION **39**

APPENDICES **40**
A.1 APPENDIX 1 TITLE 40
A.2 APPENDIX 2 TITLE 40

BIBLIOGRAPHY **41**

LIST OF FIGURES

1.1 Nombre de nouveaux secrets détectés sur GitHub entre 2023 et 2024 5
1.2 Le trilemme de la sécurité dans les infrastructures programmables 7
1.3 Adaptation du processus 2TUP au projet CloudSentinel 11

2.1 Pipeline DevSecOps : intégration Shift-Left/Right 17
2.2 Approche Shift-Left (contrôles précoces) et Shift-Right (surveillance en production) 19
2.3 Architecture logique du Policy-as-Code (modèle PDP/PEP) 23
2.4 Open Policy Agent : architecture de découplage politique/décision 25
2.5 Chaîne de traitement DevSecOps : de la détection à la décision 36

LIST OF TABLES

1.1 Lien entre vecteurs de risques et outils de détection de CloudSentinel 6
1.2 Trilemme sécurité Cloud : tensions mesurables du marché 8
1.3 Positionnement de CloudSentinel face aux solutions du marché 9

2.1 Comparaison DevOps vs DevSecOps d'après [7] 18
2.2 Couverture SDLC : Shift-Left/Right et KPI DevSecOps [8] 20
2.3 Les quatre métriques DORA et leur application dans une approche DevSecOps [9, 10]. 21
2.4 Comparaison synthétique des principales catégories d'outils DevSecOps 33

LIST OF ABBREVIATIONS

GENERAL INTRODUCTION

Chapter

1 Contexte Industriel et Enjeux de la Sécurité Cloud

Contents

1.1 Introduction **3**

1.2 Organisation d'accueil **3**

1.3 Le paradigme de l'infrastructure programmable **3**

1.3.1 Convergence du Cloud Computing et de la culture DevOps . . 3

1.3.2 Infrastructure as Code : l'infrastructure devenue logiciel 4

1.4 Les nouveaux vecteurs d'insécurité dans les environnements Cloud **4**

1.4.1 Fuite de secrets et compromission d'identités 4

1.4.2 Misconfigurations d'infrastructure as Code 5

1.4.3 Configuration Drift et perte de contrôle opérationnel 6

1.5 Problématique : le trilemme de la sécurité Cloud **7**

1.5.1 Le conflit des priorités : Vélocité vs Sécurité vs Gouvernance . 7

1.5.2 Synthèse des défis et problématique 8

1.6 Étude comparative et positionnement stratégique **8**

1.6.1 Analyse des approches actuelles 9

1.6.2 Tableau comparatif détaillé 9

1.7 Présentation du projet **9**

1.7.1 Vision du projet 10

1.7.2 Objectifs spécifiques 10

1.8 Méthodologie de travail **10**

1.8.1 Présentation du processus 2TUP 11

1.8.2 Justification du choix pour un projet DevSecOps 12

1.9 Conclusion **12**

1.1 Introduction

Le passage massif au Cloud et au DevOps a accéléré les déploiements, mais a aussi multiplié les vulnérabilités liées au code. Ce chapitre pose le cadre industriel du projet, examine les enjeux de sécurité modernes et introduit la plateforme CloudSentinel comme réponse au besoin de concilier vélocité et contrôle.

1.2 Organisation d'accueil

1.3 Le paradigme de l'infrastructure programmable

Le paradigme de l'infrastructure programmable marque une rupture avec les modèles d'administration traditionnels. Les ressources informatiques, calcul, réseau, stockage et sécurité ne sont plus gérées manuellement mais définies sous forme de code et orchestrées par des outils d'automatisation, permettant un provisionnement dynamique, reproductible et versionné.

1.3.1 Convergence du Cloud Computing et de la culture DevOps

Cette convergence permet de passer d'une gestion manuelle par « tickets » à une gestion automatisée par « flux » (pipelines). Selon DORA 2025, les équipes Elite réduisent leur MTTR de 2 600h à 1h (DevOps Research and Assessment), les organisations les plus performantes utilisent cette synergie pour réduire leur délai de mise sur le marché tout en augmentant la stabilité des services [1]. Cependant, cette vélocité impose une redéfinition du périmètre de sécurité, car les contrôles manuels traditionnels deviennent le goulot d'étranglement du cycle de livraison.

1.3.2 Infrastructure as Code : l'infrastructure devenue logiciel

L'IaC est la pierre angulaire de l'infrastructure programmable. Elle consiste à définir, provisionner et gérer les ressources (réseaux, machines virtuelles, bases de données) à l'aide de fichiers de configuration versionnés (ex. : Terraform, CloudFormation). Selon Kief Morris, cette approche permet d'appliquer les pratiques de l'ingénierie logicielle à la gestion des systèmes pour garantir des changements rapides et sûrs.
Elle offre trois avantages majeurs pour le projet CloudSentinel :

1. **La Reproductibilité** : Un environnement peut être recréé à l'identique, limitant les erreurs humaines liées aux configurations manuelles.
2. **Le Versionnement** : Chaque modification est tracée dans un dépôt Git, offrant un historique complet des changements (Audit Trail).
3. **L'Auditabilité pré-déploiement** : Puisque l'infrastructure est du code, elle peut être analysée par des outils de scan statique (Shift-Left) avant même d'être déployée [2].

Toutefois, l'IaC introduit un risque systémique : une faille de sécurité dans un script Terraform peut être répliquée sur des centaines de ressources en quelques secondes, rendant l'automatisation de la sécurité (DevSecOps) impérative.

1.4 Les nouveaux vecteurs d'insécurité dans les environnements Cloud

1.4.1 Fuite de secrets et compromission d'identités

L'automatisation et l'abstraction offertes par le Cloud modifient profondément la surface d'attaque. Si l'erreur humaine reste le facteur principal, sa codification via l'IaC lui confère une portée systémique.

Dans une architecture Cloud-Native, l'identité constitue désormais le nouveau périmètre de sécurité. Les secrets (clés d'API, jetons OAuth, credentials) représentent les éléments centraux

permettant l'accès aux ressources cloud. Le phénomène de *Secrets Sprawl* (dispersion des données d'accès sensibles dans les dépôts de code) est devenu un risque majeur.

L'ampleur du problème est alarmante : selon GitGuardian 2025, **23,8 millions de nouveaux secrets** détectés sur GitHub en 2024 (+25% vs 2023)[3], illustrant une croissance continue de l'exposition des identifiants sensibles dans les dépôts publics.

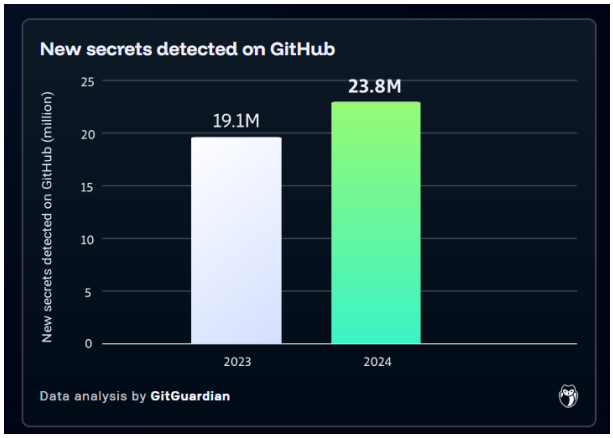

Figure 1.1: Nombre de nouveaux secrets détectés sur GitHub entre 2023 et 2024

⇒ Cette tendance souligne l'incapacité des pratiques de gestion des identités traditionnelles à suivre le rythme des pipelines DevOps et renforce la nécessité d'intégrer des mécanismes automatisés de détection de secrets dans les chaînes CI/CD.

Une fois « hardcodés » dans les scripts d'infrastructure ou les Dockerfiles, ces identifiants critiques, souvent liés à des identités non-humaines (*Non-Human Identities*), sont persistés dans l'historique Git. Un attaquant peut alors procéder à une élévation de privilèges ou à une exfiltration de données massives. La détection de ces fuites nécessite une analyse de l'historique des commits via des outils d'Entropy Scanning (détection de chaînes aléatoires) et de Regex-based detection (signatures connues) intégrés au pipeline.

1.4.2 Misconfigurations d'infrastructure as Code

Les erreurs de configuration sont la première cause de brèches de sécurité dans le Cloud. Contrairement aux vulnérabilités logicielles classiques (CVE), les misconfigurations résultent de l'application de paramètres non sécurisés lors du provisionnement des ressources.

Selon SentinelOne (2026), plus de 31% des brèches cloud proviennent de misconfigurations et erreurs manuelles [4]. Pour répondre à ce défi, CloudSentinel s'appuie sur des outils de scan statique pour valider la conformité par rapport aux CIS Benchmarks :

Risque Identifié	**Exemple Technique**	**Outil Associé**
Exposition de stockage	S3 Bucket / Azure Blob public	**Checkov**
Réseaux permissifs	Port 22 (SSH) ouvert sur 0.0.0.0/0	**Checkov**
Images vulnérables	Librairies obsolètes dans un conteneur	**Trivy**
Permissions excessives	Rôles IAM ne respectant pas le moindre privilège	**Checkov**

Table 1.1: Lien entre vecteurs de risques et outils de détection de CloudSentinel

⇒ L'intégration de Checkov et Trivy répond directement à ce risque en automatisant l'audit de conformité (CIS Benchmarks) avant même le provisionnement de l'infrastructure, bloquant ainsi les misconfigurations dès le pipeline CI/CD.

1.4.3 Configuration Drift et perte de contrôle opérationnel

Le **Configuration Drift** (dérive de configuration) désigne la divergence entre l'état déclaré de l'infrastructure et sa réalité opérationnelle. Ce phénomène compromet l'idempotence des déploiements et provient de plusieurs facteurs critiques :

- **Modifications "Out-of-band" :** Actions manuelles effectuées directement sur le portail cloud (*ClickOps*) pour résoudre une urgence, contournant le cycle de vie *GitOps*.
- **Dynamisme du Runtime :** Changements induits par l'autoscaling, les mises à jour automatiques de certificats ou le *patching* dynamique des instances qui ne sont pas reflétés dans le *Terraform State*.
- **Érosion de la posture :** Accumulation de petits changements non tracés qui, à terme, créent des failles de sécurité invisibles pour les outils d'audit statique.

1.5 Problématique : le trilemme de la sécurité Cloud

L'analyse des vecteurs de risques met en lumière un paradoxe structurel au sein des organisations modernes. La transition vers le modèle DevSecOps soulève un défi complexe que l'on peut qualifier de « trilemme de la sécurité Cloud ». Ce trilemme oppose trois impératifs souvent contradictoires : la Vélocité, la Sécurité et la Gouvernance.

1.5.1 Le conflit des priorités : Vélocité vs Sécurité vs Gouvernance

Le trilemme de la sécurité Cloud oppose trois impératifs souvent contradictoires. Chaque paire de priorités crée une zone de tension spécifique (voir Figure 1.2) :

- **Vélocité vs Sécurité :** L'automatisation des déploiements rapides se heurte aux contrôles de sécurité traditionnels, souvent perçus comme des freins.
- **Sécurité vs Gouvernance :** La multiplication d'outils de scan dispersés empêche une vision centralisée et une corrélation efficace des risques.
- **Gouvernance vs Vélocité :** Le besoin d'auditabilité et de conformité stricte entre en conflit avec l'agilité et le rythme élevé des pipelines CI/CD.

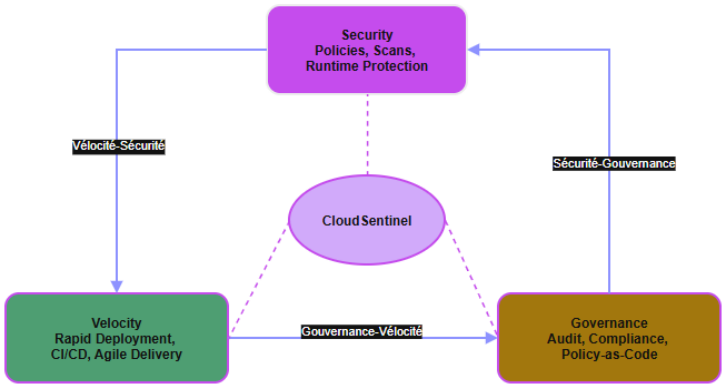

Figure 1.2: Le trilemme de la sécurité dans les infrastructures programmables

Les études DORA et SentinelOne mettent en évidence les tensions structurelles du trilemme sécurité Cloud [1, 4]. Ces métriques illustrent parfaitement pourquoi les approches traditionnelles échouent à concilier ces impératifs antagonistes [1] :

Conflit	**Impact**	**Conséquence**
Vélocité vs Sécurité	Scans = goulot d'étranglement	Lead Time $\times 6$ [1]
Sécurité vs Gouvernance	Outils éclatés et silos	47% faux positifs [4]
Gouvernance vs Vélocité	Audits manuels chronophages	MTTR $>$ 30j [1]

Table 1.2: Trilemme sécurité Cloud : tensions mesurables du marché

$\Rightarrow$ Ces métriques illustrent parfaitement pourquoi les approches traditionnelles échouent.

1.5.2 Synthèse des défis et problématique

L'analyse montre que les approches traditionnelles échouent à :

1. **Centraliser la décision** via un *Policy Decision Point* (PDP) unique pour tous les scanners.
2. **Gérer les exceptions** de manière auditable avec un mécanisme de *Risk Acceptance* versionné.
3. **Corréler le Code et le Runtime** pour assurer une continuité entre la prévention (Shift-Left) et la surveillance (Shift-Right).

1.6 Étude comparative et positionnement stratégique

Afin de situer **CloudSentinel** dans l'écosystème actuel de la cybersécurité, nous avons analysé trois solutions représentatives : deux leaders commerciaux (**Wiz** et **Prisma Cloud**) et l'alternative open source la plus proche de notre démarche (**Nubicustos**). Cette étude se base sur des critères de coût, de transparence et de capacité d'orchestration.

1.6.1 Analyse des approches actuelles

- **Wiz et Prisma Cloud (Solutions CNAPP) :** Ce sont des plateformes « tout-en-un » extrêmement puissantes mais fonctionnant comme des « boîtes noires » propriétaires. Leurs algorithmes de corrélation et de décision sont opaques, empêchant toute auditabilité du moteur de sécurité. De plus, leur coût prohibitif et le phénomène de *Vendor Lock-in* les rendent inadaptées pour des organisations cherchant une souveraineté totale.
- **Nubicustos (Alternative Open Source) :** C'est un agrégateur de données performant. Cependant, il se concentre sur l'observation et l'intelligence (graphe d'attaque) sans proposer de *Quality Gate* bloquante native en CI/CD, ni de remédiation active automatisée.

1.6.2 Tableau comparatif détaillé

Critères	**Wiz**	**Prisma Cloud**	**Nubicustos**	**CloudSentinel**
Type / Licence	Propriétaire	Propriétaire	Open Source	**Open Source**
Transparence	Boîte noire	Boîte noire	Auditable	**Auditable (100%)**
Shift-Left (CI/CD)	Complet	Complet	Pas de Gate	**Gate OPA (DENY)**
Shift-Right (Drift)	Temps réel	CSPM intégré	Audit	**Actif (Drift Engine)**
Remédiation	Automatique	Automatique	Guidance	**Graduée (Custodian)**
Gouvernance	Console SaaS	Console SaaS	UI PostgreSQL	**ASPM (DefectDojo)**
Coût	Très élevé	Très élevé	Gratuit	**Gratuit**

Table 1.3: Positionnement de CloudSentinel face aux solutions du marché

1.7 Présentation du projet

Le projet **CloudSentinel** est conçu comme une plateforme de gouvernance sécurité unifiée, visant à automatiser la protection des infrastructures cloud tout au long de leur cycle de vie. Sa philosophie repose sur le passage d'une sécurité « réactive » à une sécurité « intégrée », résumée par son slogan : *« De la prévention dans le code à la gouvernance continue en production »*.

1.7.1 Vision du projet

La vision de CloudSentinel est de réconcilier le trilemme Sécurité-Vélocité-Gouvernance en orchestrant des outils open source de référence au sein d'une architecture unique.
L'idée centrale est de ne plus traiter la sécurité comme une étape isolée, mais comme un flux ininterrompu qui accompagne la ressource cloud depuis sa définition dans le code (**Shift-Left**) jusqu'à son cycle de vie en production (**Shift-Right**).

1.7.2 Objectifs spécifiques

La vision de CloudSentinel est de fournir une solution souveraine et *Open Source* capable d'orchestrer la sécurité sans sacrifier la vélocité des équipes DevOps. Les principaux objectifs sont :

- **Unification :** Centraliser les résultats de multiples scanners de sécurité (secrets, IaC, vulnérabilités) dans un format normalisé.
- **Automatisation de la décision :** Remplacer les validations manuelles par une *Quality Gate* automatisée basée sur le moteur de politiques *Open Policy Agent* (OPA).
- **Traçabilité intégrale :** Assurer que chaque vulnérabilité détectée, chaque risque accepté et chaque remédiation effectuée soit historisé dans *DefectDojo* pour des besoins d'audit.
- **Souveraineté :** Maintenir le contrôle total sur la logique de sécurité et les données de vulnérabilité, sans dépendance vis-à-vis d'éditeurs tiers.

1.8 Méthodologie de travail

Pour mener à bien la conception et la réalisation de la plateforme **CloudSentinel**, nous avons adopté le processus de développement **2TUP (Two Track Unified Process)**. Cette approche, issue de l'*Unified Process*, se distingue par son organisation en « Y », permettant de gérer parallèlement les évolutions fonctionnelles et les contraintes techniques.

1.8.1 Présentation du processus 2TUP

Le cycle de vie du processus 2TUP s'articule autour de trois phases majeures (voir Figure 1.3) :

- **La branche fonctionnelle (Axe métier) :** Elle se concentre sur la capture des besoins et la définition de la « logique de sécurité ». Dans le cadre de ce projet, elle inclut la modélisation des politiques de conformité (*Policy-as-Code*), indépendamment de l'infrastructure cible.
- **La branche technique (Axe architecture) :** Elle examine les contraintes d'implémentation et le socle technologique (Cloud Azure/AWS, pipelines CI/CD, formats de données JSON). Elle permet de construire un prototype technique capable de supporter l'orchestration des différents scanners.
- **La phase de conception et réalisation :** C'est le point de fusion des deux branches (le « mariage »), où les politiques de sécurité métier sont injectées et automatisées sur l'architecture technique validée pour produire le système final.

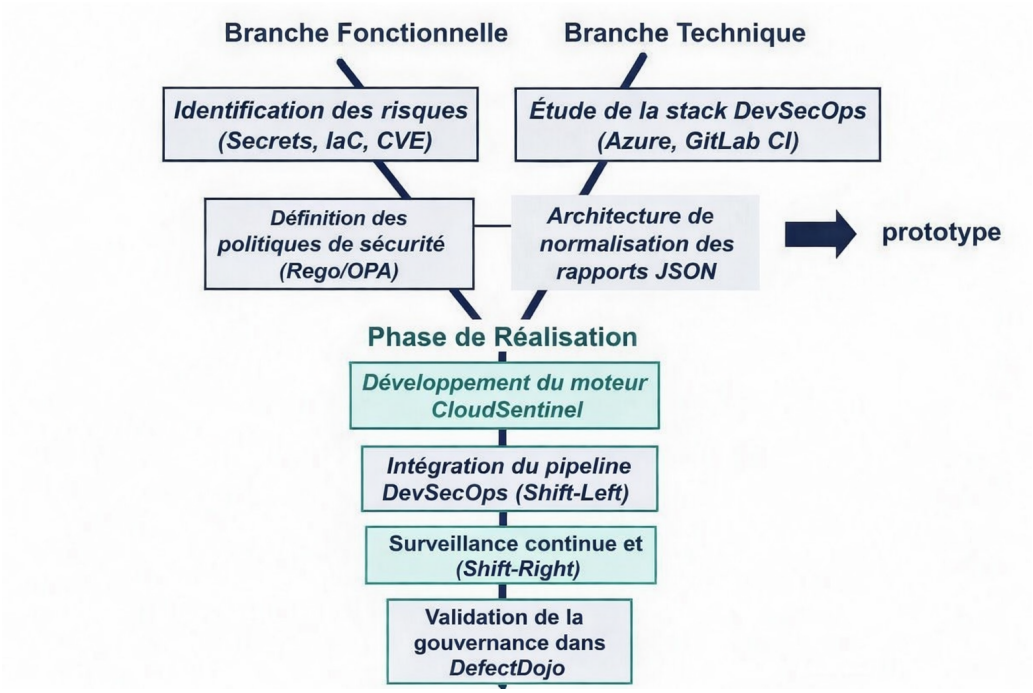

Figure 1.3: Adaptation du processus 2TUP au projet CloudSentinel

1.8.2 Justification du choix pour un projet DevSecOps

L'adoption de la méthodologie 2TUP pour CloudSentinel ne relève pas d'un choix arbitraire, mais répond à des besoins critiques spécifiques à l'ingénierie de sécurité cloud :

1. **Découplage de la logique de sécurité (Agnosticisme) :** Un avantage majeur du 2TUP est la séparation de la logique métier (branche fonctionnelle) de l'infrastructure (branche technique). Cela permet à CloudSentinel de définir des politiques *Policy-as-Code* universelles qui restent valides même si l'architecture technique évolue (ex: passage de Microsoft Azure à AWS).
2. **Gestion de l'hétérogénéité des outils :** Notre projet repose sur l'orchestration d'outils disparates (*Gitleaks, Checkov, Trivy, OPA*). La branche technique permet de stabiliser précocement l'architecture de **normalisation des données JSON**, une étape cruciale pour garantir l'interopérabilité avant l'implémentation des règles de blocage.
3. **Maîtrise des risques d'intégration :** Le modèle DevSecOps impose une intégration transparente dans les pipelines CI/CD. En traitant les contraintes techniques (performances des scans, limites des APIs Cloud) dès la branche technique, nous minimisons les risques de ralentissement de la vélocité DevOps lors de la phase de réalisation.
4. **Auditabilité et Traçabilité :** La rigueur documentaire imposée par le 2TUP assure une traçabilité intégrale entre les besoins de gouvernance identifiés au départ et les mécanismes techniques de remédiation mis en place à l'arrivée.

1.9 Conclusion

Ce premier chapitre a permis de poser le cadre du projet **CloudSentinel** en identifiant les enjeux critiques de la sécurité Cloud et en définissant nos objectifs de gouvernance. Après avoir établi nos besoins et la méthodologie **2TUP** adoptée, nous consacrerons le chapitre suivant à l'étude du **Contexte et État de l'Art du DevSecOps** afin d'approfondir les concepts et technologies qui constitueront le socle de notre solution.

Chapter 2

État de l'Art : Sécurité Cloud et Technologie DevSecOps

Contents

2.1 Introduction 15

2.2 Sécurité du Cloud : modèle de responsabilité et Zero Trust . 15

2.2.1 Le modèle de responsabilité partagée 15

2.2.2 Zero Trust : principe structurant de la gouvernance cloud . . . 16

2.3 Évolution vers le DevSecOps 17

2.3.1 Du DevOps au DevSecOps 17

2.3.2 Shift-Left et Shift-Right Security 18

2.3.3 Métriques DORA appliquées à la sécurité 20

2.4 Policy-as-Code : Modélisation avancée de la décision 21

2.4.1 Définition et principes fondamentaux 21

2.4.2 Architecture PDP/PEP : Un système distribué critique 22

2.4.3 Open Policy Agent (OPA) et l'écosystème 24

2.4.4 Cas d'usage, avantages et limites 25

2.5 Écosystème d'outils DevSecOps : analyse comparative 27

2.5.1 Catégorisation fonctionnelle 27

2.5.2 Outils Shift-Left 28

2.5.3 Moteurs de Policy-as-Code 29

2.5.4 Outils Shift-Right 30

2.5.5 Plateformes transverses de gestion des findings 32

2.5.6 Synthèse comparative 32

2.6 Architecture d'orchestration des contrôles de sécurité 34

2.6.1 La crise de la normalisation et les contrats de données 34

2.6.2 Chaîne de traitement d'un pipeline DevSecOps 35

2.6.3 Enjeux de passage à l'échelle (Scalabilité et Performance) . . . 36

2.7 Synthèse critique : lacunes dictant l'architecture cible 37
2.8 Conclusion 38

2.1 Introduction

Le chapitre précédent a établi le contexte industriel : vecteurs d'insécurité cloud et trilemme **Vélocité-Sécurité-Gouvernance**. Ce chapitre présente l'état de l'art DevSecOps : concepts (Zero Trust, Shift-Left/Right), mécanismes (Policy-as-Code, architecture PDP/PEP), outils spécialisés, et enjeux d'orchestration multi-scanners. Une synthèse critique des limites actuelles prépare la conception de la plateforme CloudSentinel.

2.2 Sécurité du Cloud : modèle de responsabilité et Zero Trust

2.2.1 Le modèle de responsabilité partagée

Le Cloud Computing introduit un partage des responsabilités de sécurité entre le fournisseur et le client, formalisé par le **Shared Responsibility Model**. Le fournisseur assure la sécurité *du* cloud (infrastructure physique, hyperviseur, réseau global), tandis que le client est responsable de la sécurité *dans* le cloud : configurations, identités, données et applications [5]. Cette délimitation a une conséquence directe : la grande majorité des incidents de sécurité cloud résulte de failles côté client — misconfigurations, permissions excessives et secrets exposés — et non de failles dans l'infrastructure du fournisseur. Le chapitre précédent a documenté ces vecteurs en détail (secrets sprawl, misconfigurations IaC, drift de configuration) ; leur traitement technique par les outils DevSecOps fait l'objet des sections suivantes. Trois propriétés spécifiques aux environnements cloud conditionnent le choix des approches de sécurité :

- **Conformité éphémère** : une architecture validée au déploiement (*Day-0*) peut devenir vulnérable quelques heures plus tard, sous l'effet de nouvelles CVE ou de modifications manuelles. La sécurité cloud est donc un *processus continu*, non un état statique.
- **Périmètre dynamique** : les ressources apparaissent et disparaissent automatiquement, rendant les approches de sécurité basées sur un périmètre réseau fixe structurellement inadaptées.

- **Surface d'attaque codifiée** : puisque l'infrastructure est définie par du code (IaC), une faille dans un script Terraform peut se propager instantanément sur des centaines de ressources. L'automatisation de la sécurité n'est plus optionnelle : elle devient impérative.

2.2.2 Zero Trust : principe structurant de la gouvernance cloud

Face à la disparition du périmètre réseau traditionnel, le modèle **Zero Trust** s'est imposé comme le cadre conceptuel de référence : *ne jamais faire confiance, toujours vérifier* [6]. Ce modèle repose sur trois principes fondamentaux :

- **Vérification systématique** : aucune ressource, utilisateur ou service n'est considéré comme sûr par défaut, quelle que soit sa localisation réseau. Chaque déploiement et chaque action doit être explicitement authentifié, autorisé et validé.
- **Principe du moindre privilège** : chaque entité (utilisateur, service, pipeline) ne dispose que des droits strictement nécessaires à son rôle, limités dans le temps et dans le périmètre.
- **Hypothèse de compromission** : le système est conçu en supposant qu'une intrusion est possible à tout moment. La priorité est donnée à la détection rapide, la contenance des accès et la traçabilité complète.

Dans un contexte DevSecOps, le Zero Trust se traduit par trois exigences opérationnelles structurantes pour l'architecture CloudSentinel :

1. **Valider tout artefact avant déploiement** (Shift-Left) : aucun code, fichier IaC ou image de conteneur ne peut être déployé sans validation automatisée par des politiques formelles.
2. **Vérifier continuellement la posture en production** (Shift-Right) : l'état réel de l'infrastructure doit être audité de manière régulière et comparé à l'état attendu pour détecter les dérives.
3. **Centraliser la décision via des politiques auditables** (Policy-as-Code) : la logique d'autorisation doit être formalisée, versionnée et traçable, et non dispersée dans des configurations manuelles ou des règles implicites.

Ces trois exigences constituent le fil conducteur de l'architecture DevSecOps présentée dans ce chapitre et fondent les choix de conception de la plateforme CloudSentinel.

2.3 Évolution vers le DevSecOps

2.3.1 Du DevOps au DevSecOps

Le DevOps, apparu vers 2009, a rapproché le développement et l'exploitation grâce à l'automatisation des pipelines CI/CD et à une culture collaborative. Cependant, dans ce modèle, la sécurité restait souvent traitée tardivement, ce qui générait des retards et des vulnérabilités en production. À partir de 2015, le **DevSecOps** intègre la sécurité *dès la conception* en favorisant la collaboration entre les équipes de développement, de sécurité et d'exploitation, et en automatisant les contrôles de sécurité dans les pipelines CI/CD. L'émergence du **Policy-as-Code** (notamment avec Open Policy Agent et le langage Rego) à partir de 2020 constitue une avancée majeure de cette intégration [7]. Cette approche réduit significativement le **MTTR** (Mean Time To Remediate) et constitue la réponse opérationnelle au trilemme Vélocité-Sécurité-Gouvernance documenté au chapitre précédent.

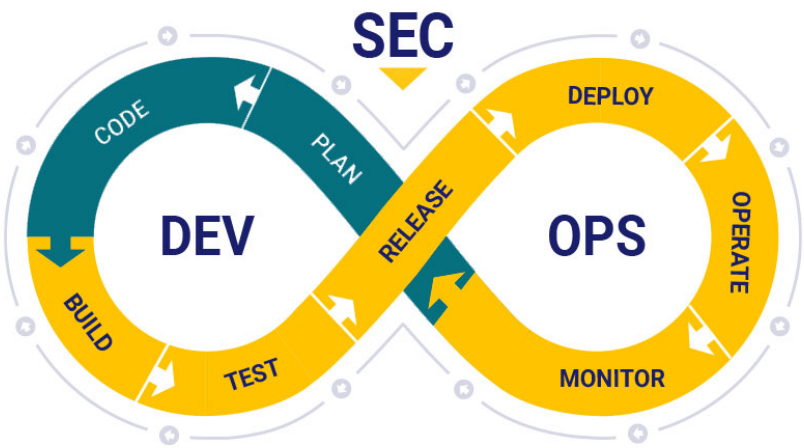

Figure 2.1: Pipeline DevSecOps : intégration Shift-Left/Right

Le tableau 2.1 présente les principales différences entre DevOps et DevSecOps.

Aspect	DevOps	DevSecOps
Objectif principal	Vitesse, collaboration, livraison efficace	Sécurité intégrée à la vitesse et à la qualité
Phase sécurité	Souvent après déploiement	Intégrée dès la conception (Shift-Left)
Équipes impliquées	Développeurs et Opérations	Développeurs, Sécurité, Opérations (collaboration continue)
Approche tests	Automatisation CI/CD pour releases rapides	Tests de sécurité automatisés (SAST, SCA) intégrés au pipeline CI/CD
Résultat	Livraison logicielle rapide	Logiciel sécurisé, conforme et livré rapidement

Table 2.1: Comparaison DevOps vs DevSecOps d'après [7]

2.3.2 Shift-Left et Shift-Right Security

L'intégration de la sécurité dans le cycle de vie logiciel (**SDLC**) nécessite une approche hybride combinant prévention précoce et surveillance continue, représentée par le déplacement des contrôles le long de la chaîne de valeur.

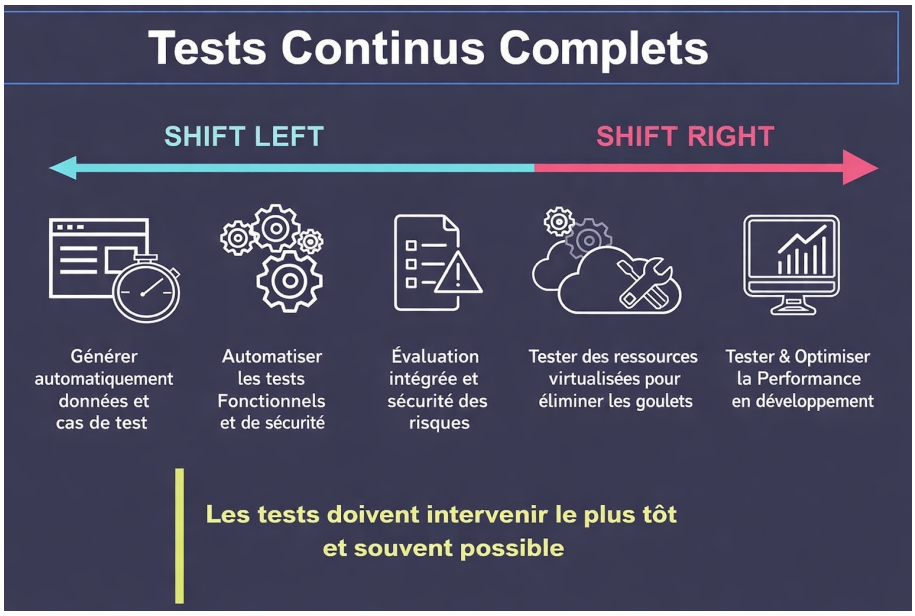

Figure 2.2: Approche Shift-Left (contrôles précoces) et Shift-Right (surveillance en production)

2.3.2.1 Shift-Left Security

Cette approche proactive permet de corriger les vulnérabilités dès la phase de développement, minimisant le coût de remédiation. Les contrôles Shift-Left incluent :

- **SAST** (*Static Application Security Testing*) : analyse statique du code pour détecter vulnérabilités et patterns dangereux.
- **Secrets Scanning (SCS)** : détection des informations sensibles exposées dans le code source et l'historique Git via des expressions régulières et validation cryptographique.
- **IaC Validation** : conformité des templates Terraform/CloudFormation avec les standards de sécurité (CIS, NIST) avant déploiement.
- **SCA et SBOM** : analyse des dépendances et génération du *Software Bill of Materials* pour la gestion des vulnérabilités tierces.

2.3.2.2 Shift-Right Security

Le Shift-Right complète la sécurité par surveillance et remédiation en production :

- **Drift Detection** : détection des divergences entre état souhaité (IaC) et état réel de l'infrastructure, identifiant les modifications manuelles (*out-of-band changes*).
- **CSPM** (*Cloud Security Posture Management*) : évaluation continue de la conformité des ressources cloud déployées.
- **RASP** et **WAF** : protection dynamique des applications en cours d'exécution.
- **Remédiation automatisée** : actions correctives déclenchées par politique (tagging, isolement, suppression).

Le tableau 2.2 synthétise les phases du SDLC, les contrôles associés et les KPI correspondants.

Phase	**Shift-Left**	**Shift-Right**	**Outils**	**KPI / Contrôle**
Code	SAST, SCS	—	SonarQube, Gitleaks	MTTR < 1h, 0 secret
IaC	Checkov, TFLint	Drift Detection	OPA Gatekeeper, Terrascan	100 % conformité IaC
Build	SCA, SBOM	—	CycloneDX, Dependency-Track	CVSS > 7 = BLOCK
Runtime	—	RASP, WAF, CSPM	Falco, Tetragon, Prowler	99,9 % uptime

Table 2.2: Couverture SDLC : Shift-Left/Right et KPI DevSecOps [8]

2.3.3 Métriques DORA appliquées à la sécurité

Les métriques DORA (DevOps Research and Assessment) constituent aujourd'hui le standard reconnu pour mesurer la performance des processus de livraison logicielle. Elles ont été étendues au DevSecOps pour mesurer simultanément la **vélocité**, la **résilience** et la **conformité sécurité**, tout en garantissant que l'intégration de contrôles automatisés ne dégrade pas la cadence des déploiements [10, 11].

Ces indicateurs permettent de piloter des pipelines CI/CD sécurisés, en garantissant que les scans SAST, SCA, IaC et le secret scanning s'exécutent rapidement, que les politiques de

sécurité sont respectées automatiquement (Policy-as-Code), et que les mécanismes de réponse aux incidents limitent le temps d'exposition aux vulnérabilités.

Métrique DORA	**Définition**	**Elite / High Performer**	**Low Performer**	**Exemple de contrôle DevSecOps**
Deployment Frequency (DF)	Fréquence des déploiements en production	Plusieurs fois par jour	1 fois par mois ou moins	Policy-as-Code (PaC) : vérification automatisée des politiques avant déploiement
Lead Time for Changes (LT)	Temps entre le commit et la mise en production	Moins d'un jour	1 semaine à plusieurs semaines	Shift-Left Security : scans SAST, SCA, IaC et secret scanning intégrés dans CI/CD
Change Failure Rate (CFR)	Pourcentage de déploiements provoquant un échec ou rollback	0–15%	46–60%	Security Gates : blocage automatique si vulnérabilités critiques (CVSS > 7.0)
Mean Time to Recovery (MTTR)	Temps nécessaire pour restaurer le système après un incident	Moins d'une heure	Plusieurs heures à 1 jour	Incident Response Automation : rollback automatique et détection de dérive (drift)

Table 2.3: Les quatre métriques DORA et leur application dans une approche DevSecOps [9, 10].

Ces métriques fournissent un cadre quantifiable pour mesurer la maturité DevSecOps et pour aligner la sécurité avec la vitesse et la résilience des pipelines CloudSentinel. Elles permettent également de détecter les goulots de sécurité, de mesurer l'efficacité des contrôles automatisés et d'optimiser la gouvernance et la traçabilité de l'infrastructure cloud.

2.4 Policy-as-Code : Modélisation avancée de la décision

2.4.1 Définition et principes fondamentaux

Le **Policy-as-Code** (PaC) désigne une approche de gouvernance dans laquelle les politiques de sécurité, de conformité et de contrôle opérationnel sont formalisées sous forme de règles exécutables, déclaratives, versionnées et testables. Une politique n'est plus un document statique à lecture humaine, mais un artefact technique interprété automatiquement par un moteur de décision [12, 14].

Cette approche répond aux limites des mécanismes de contrôle manuels (audits ponctuels, checklists statiques) qui s'adaptent difficilement à la fréquence élevée des changements cloud et introduisent des délais incompatibles avec le déploiement continu. Le PaC exprime explicitement les contraintes de sécurité et les fait évaluer de manière systématique à différents points de contrôle : pipelines CI/CD, déploiements IaC et environnements d'exécution cloud. Les principes structurants du Policy-as-Code sont :

- **Formalisation explicite des règles** : les exigences de sécurité sont traduites en règles formelles, réduisant l'ambiguïté d'interprétation.
- **Versionnement et testabilité** : les politiques sont gérées comme des artefacts logiciels, soumises à revue et contrôle de version.
- **Automatisation des décisions** : évaluation répétable et cohérente à partir de données d'entrée structurées.
- **Intégration au cycle DevSecOps** : application dans les chaînes CI/CD, l'infrastructure et les environnements d'exécution.
- **Traçabilité et auditabilité** : chaque décision est reliée à une entrée, à une règle et à un contexte d'exécution identifiable.

2.4.2 Architecture PDP/PEP : Un système distribué critique

Dans la littérature et les implémentations industrielles, les systèmes de PaC s'appuient sur une architecture logique composée de cinq éléments complémentaires, illustrée par la Figure 2.3. Cette architecture repose sur le standard XACML (eXtensible Access Control Markup Language) et sépare strictement la décision de l'exécution.

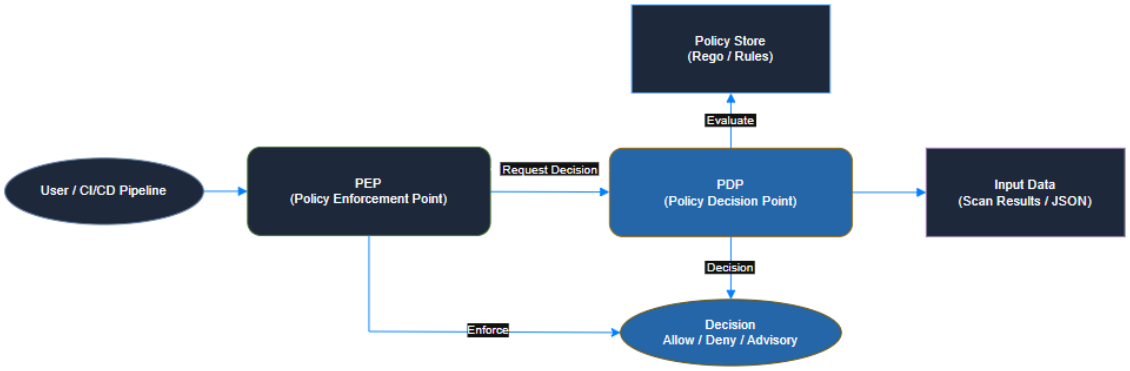

Figure 2.3: Architecture logique du Policy-as-Code (modèle PDP/PEP)

- **Policy Decision Point (PDP)** : composant chargé d'évaluer les politiques à partir d'un ensemble de faits et de produire une décision (*allow*, *deny* ou *advisory*).
- **Policy Enforcement Point (PEP)** : composant consommant la décision produite par le PDP et appliquant l'action correspondante (blocage de pipeline, rejet d'un déploiement, émission d'une alerte).
- **Input / Data** : données soumises à évaluation — résultats de scanners, manifestes Kubernetes, fichiers IaC ou métadonnées de pipeline.
- **Policy Store** : dépôt centralisé et versionné contenant les politiques ainsi que les données complémentaires nécessaires à leur exécution.
- **Decision Output** : sortie structurée contenant le verdict final, les motifs de décision, les violations détectées, les exceptions et des métriques d'audit.

Cependant, dans un contexte d'entreprise cloud-native, le **Policy Decision Point (PDP)** n'est pas un simple évaluateur de règles locales. Il s'agit d'un **système distribué critique** nécessitant des mécanismes avancés :

- **Policy Layering Composition** : l'évaluation d'un artefact ne dépend rarement d'une seule règle. Le PDP doit fusionner des règles globales (définies par le RSSI) et des règles locales (définies par les équipes produits). Cette composition (*multi-policy evaluation*) exige une résolution fine des conflits (ex : un *deny* global écrase un *allow* local).

- **Context Injection** : une décision de sécurité ne peut souvent pas se prendre uniquement sur la base du payload analysé. Le PDP doit être capable d'enrichir l'évaluation de contexte externe (ex : "est-ce que ce dépôt correspond à une application critique (Tier 1) exposée sur Internet ?").

- **Découplage strict** : le **Policy Enforcement Point (PEP)** (le pipeline GitLab/GitHub) doit interroger le PDP de manière asynchrone ou via des API extrêmement résilientes, sous peine de bloquer toute la chaîne de production logicielle en cas de latence.

L'intérêt principal de cette architecture est la **séparation de la logique de décision et de la logique d'exécution** : les politiques peuvent évoluer indépendamment des outils ou des plateformes sur lesquels elles sont appliquées.

2.4.3 Open Policy Agent (OPA) et l'écosystème

Plusieurs solutions implémentent ce paradigme, avec des périmètres et niveaux d'intégration différents [14] :

- **Open Policy Agent (OPA)** : moteur généraliste évaluant des règles exprimées en langage déclaratif *Rego* sur des données JSON. Son caractère universel favorise l'intégration dans des contextes variés : pipelines CI/CD, IaC, microservices, Kubernetes. La puissance d'OPA réside dans sa neutralité : il ne connaît pas le domaine métier, il n'évalue que des structures de données abstraites. Cette flexibilité met en exergue un défi architectural majeur : OPA n'est fiable que si la donnée JSON qu'on lui fournit (l'Input) respecte un schéma strict et prédictible.

- **HashiCorp Sentinel** : moteur intégré à l'écosystème HashiCorp (Terraform Enterprise, Vault, Consul), principalement utilisé pour la gouvernance d'infrastructure.

- **Kyverno** : solution orientée Kubernetes exprimant les politiques en YAML, adaptée à la validation, mutation ou génération de ressources sans langage de politique distinct.

- **Gatekeeper** : contrôleur d'admission basé sur OPA pour Kubernetes, centré sur le *cluster admission control*.

- **Conftest** : outil en ligne de commande s'appuyant sur OPA pour évaluer localement (ou en pipeline) des fichiers de configuration YAML, JSON, Dockerfile ou Terraform plan.

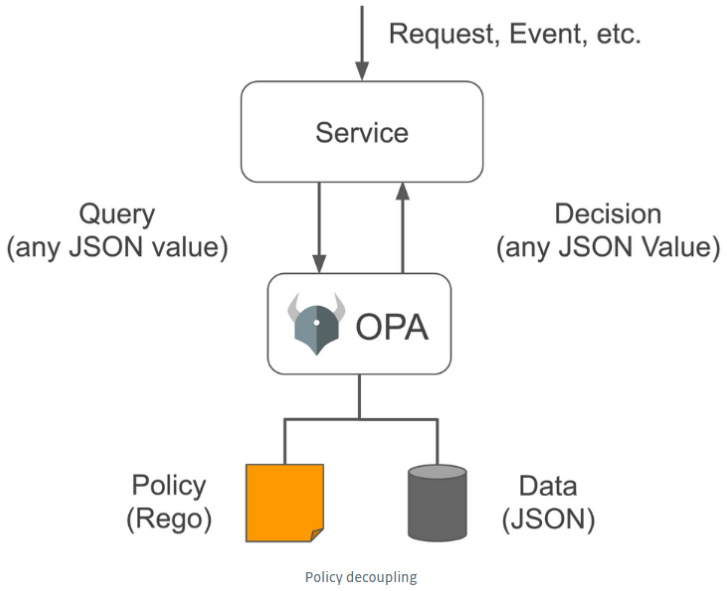

Figure 2.4: Open Policy Agent : architecture de découplage politique/décision

2.4.4 Cas d'usage, avantages et limites

2.4.4.1 Cas d'usage en DevSecOps et cloud

- **Contrôle des configurations IaC** : validation de fichiers Terraform, CloudFormation ou manifestes Kubernetes avant déploiement (ressources exposées publiquement, absence de chiffrement, permissions excessives).
- **Contrôle d'admission Kubernetes** : refus des configurations non conformes lors de la création ou mise à jour d'objets (conteneurs privilégiés, images non approuvées, RBAC trop permissif).
- **Gouvernance des pipelines CI/CD** : blocage de builds ou déploiements contenant des secrets, des vulnérabilités critiques ou des artefacts non conformes.
- **Validation de conformité cloud** : vérification automatisée des bonnes pratiques AWS/Azure/GCP (journalisation, chiffrement, segmentation réseau).

2.4.4.2 Avantages

- **Centralisation de la gouvernance** : les règles sont définies dans une couche dédiée et réutilisables sur plusieurs projets, équipes et environnements.
- **Cohérence des décisions** : l'automatisation réduit les écarts d'interprétation et limite la variabilité des contrôles manuels.
- **Traçabilité et auditabilité** : les décisions produites par le moteur sont justifiables, rejouables et auditables.
- **Alignement DevSecOps** : intégration naturelle aux chaînes CI/CD, aux pratiques IaC et aux plateformes cloud-native.

2.4.4.3 Limites

- **Complexité des langages de politique** : l'écriture de règles avancées en Rego peut devenir difficile à maintenir, en particulier avec des contextes multiples, des exceptions ou des données hétérogènes.
- **Dépendance à la qualité des données d'entrée** : une politique n'est pertinente que si les données évaluées sont fiables et normalisées. Cette dépendance constitue un point de fragilité majeur dans les architectures multi-scanners.
- **Gestion du cycle de vie** : les politiques doivent elles-mêmes être testées, versionnées, documentées et dépréciées.
- **Risque de rigidité opérationnelle** : des politiques trop strictes peuvent ralentir la livraison et générer des frictions avec les équipes de développement.

En définitive, l'efficacité du Policy-as-Code dépend autant du moteur utilisé que de la qualité de son intégration dans l'écosystème technique et organisationnel, et surtout de la **fiabilité des contrats de données** qui alimentent le PDP.

2.5 Écosystème d'outils DevSecOps : analyse comparative

2.5.1 Catégorisation fonctionnelle

Dans les environnements cloud-native, la sécurité repose sur la combinaison de plusieurs familles d'outils complémentaires. Ceux-ci peuvent être classés selon la nature des contrôles qu'ils réalisent et leur positionnement dans le SDLC :

- **Outils de détection de secrets** : identifient mots de passe, clés API et tokens exposés dans le code, les fichiers de configuration ou l'historique Git.
- **Outils d'analyse IaC** : vérifient les fichiers Terraform, CloudFormation, Kubernetes ou Helm afin de détecter des erreurs de configuration avant déploiement.
- **Outils d'analyse de vulnérabilités et de conteneurs** : inspectent dépendances, images et artefacts binaires pour identifier des CVE connues.
- **Moteurs de Policy-as-Code** : évaluent des règles formelles sur des données structurées pour produire des décisions de gouvernance.
- **Outils de posture et remédiation post-déploiement** : assurent l'audit de conformité cloud et, dans certains cas, la remédiation automatisée.
- **Plateformes transverses de gestion des findings** : centralisent, dédupliquent et suivent les résultats issus de plusieurs scanners.

Cette catégorisation souligne que les outils DevSecOps ne sont pas substituables : certains produisent des constats techniques, d'autres organisent la décision, la remédiation ou la gouvernance de ces constats.

2.5.2 Outils Shift-Left

2.5.2.1 Détection de secrets

La détection de secrets constitue un mécanisme de prévention précoce visant à limiter l'exposition d'identifiants sensibles. *Gitleaks* se positionne comme un outil SAST orienté détection et prévention des secrets dans les dépôts Git, avec une intégration possible en *pre-commit*, en ligne de commande et dans les pipelines CI/CD [15]. Il utilise des expressions régulières complexes et des mécanismes de validation cryptographique pour identifier les credentials exposés. *TruffleHog* est mobilisé pour des scénarios d'analyse plus approfondis, notamment la validation de secrets découverts dans l'historique complet des commits. *GitGuardian* propose un écosystème plus large, orienté plateforme et surveillance continue, avec des capacités de détection étendues au-delà de Git (Slack, Jira). Les critères de comparaison retenus pour cette catégorie sont : la rapidité d'exécution, la facilité d'intégration dans la chaîne de développement, la qualité du format de sortie (JSON structuré, SARIF) et la capacité à limiter les faux positifs via des mécanismes d'allowlist.

2.5.2.2 Analyse de l'Infrastructure as Code

L'analyse IaC vise à identifier les erreurs de configuration avant déploiement. *Checkov* prend en charge plusieurs technologies (Terraform, CloudFormation, Kubernetes, Helm, ARM Templates) et propose des politiques orientées attributs ainsi que des politiques orientées graphe, permettant d'analyser non seulement des ressources isolées, mais aussi leurs relations [16]. Cette capacité relationnelle est un point de différenciation important, car de nombreuses faiblesses de sécurité émergent d'interactions entre ressources (ex : sécurité groupes associés à des instances exposées). *Terrascan* et *tfsec* couvrent des périmètres proches, avec des différences d'expressivité des règles et de couverture technologique. Checkov se distingue par son moteur de graphe permettant d'analyser les dépendances entre ressources, essentiel pour détecter les vulnérabilités relationnelles (ex : subnet public + security group permissif).

2.5.2.3 Analyse de vulnérabilités et de conteneurs

Les scanners de cette catégorie jouent un rôle central dans la sécurisation de la chaîne d'approvisionnement logicielle. *Trivy* se présente comme un scanner unifié capable de détecter vulnérabilités et mauvaises configurations dans les dépôts de code, artefacts binaires, images de conteneurs et clusters Kubernetes [17]. Ses modes de scan permettent d'inclure plusieurs familles de contrôles : vulnérabilités (CVE), mauvaises configurations, secrets et licences, selon la cible et les options retenues [18]. Trivy se distingue par sa capacité à générer des SBOM (Software Bill of Materials) et à scanner simultanément les dépendances applicatives (SCA), les configurations système et les secrets. *Grype* et *Snyk* complètent l'offre sur des périmètres similaires, avec des différences sur la qualité de la base CVE exploitée, les formats de sortie et les intégrations disponibles. La difficulté principale dans cette catégorie ne réside pas dans la simple détection, mais dans la gestion du volume d'alertes, la priorisation des CVE selon le contexte d'exploitation (exploitabilité, criticité de l'asset) et la réduction du bruit (faux positifs).

2.5.3 Moteurs de Policy-as-Code

Les moteurs de politique occupent une place particulière dans la chaîne DevSecOps. Contrairement aux scanners, leur rôle n'est pas d'identifier directement des vulnérabilités, mais d'évaluer des règles formelles sur des données structurées et de produire une décision explicite (voir Section 2.4.2). *Open Policy Agent (OPA)* se définit comme un moteur généraliste permettant d'unifier l'application des politiques à travers l'ensemble de la pile technique [13] : applications, proxys, Kubernetes, pipelines CI/CD ou API gateways. Il évalue des règles exprimées en *Rego* sur n'importe quel document JSON. *Conftest* fournit une interface CLI basée sur le langage *Rego* d'OPA, destinée à tester des configurations structurées (YAML, JSON, Terraform) [19]. Il est particulièrement adapté aux pipelines CI/CD pour valider des fichiers de configuration avant déploiement. *Sentinel*, *Gatekeeper* et *Kyverno* s'inscrivent dans la même famille mais avec des positionnements différenciés selon leur domaine d'intégration privilégié (HashiCorp Enterprise, Kubernetes admission control, policies YAML natives).

2.5.4 Outils Shift-Right

2.5.4.1 Audit de posture cloud

Les outils d'audit post-déploiement évaluent l'état réel des ressources cloud par rapport à des référentiels de sécurité, répondant directement au défi de la **conformité éphémère** identifié en Section 2.2. *Prowler* se positionne comme une plateforme de sécurité cloud ouverte, *agentless*, destinée aux évaluations de sécurité, aux audits périodiques, au monitoring continu et à la mise en conformité avec des référentiels reconnus [20]. Son architecture modulaire lui permet d'exécuter des centaines de contrôles sur AWS, Azure et GCP, couvrant des domaines tels que la gestion des identités (IAM), la configuration réseau, le chiffrement des données au repos et en transit, la journalisation et les accès publics aux ressources. Dans le contexte DevSecOps, Prowler présente plusieurs caractéristiques distinctives par rapport aux scanners IaC classiques :

- **Évaluation de l'état réel** : contrairement à Checkov qui analyse des fichiers Terraform avant déploiement, Prowler interroge directement les API cloud pour évaluer l'infrastructure telle qu'elle existe en production, détectant ainsi les dérives et les ressources créées hors IaC.
- **Alignement multi-référentiels** : chaque contrôle peut être associé à un ou plusieurs frameworks de conformité (CIS Benchmarks, NIST 800-53, ISO 27001, SOC 2, PCI-DSS), facilitant la production de rapports d'audit exploitables par les équipes sécurité et conformité.
- **Intégration pipeline** : Prowler peut être exécuté en mode périodique (planifié) ou déclenché dans un pipeline CI/CD, permettant de valider la posture cloud après chaque déploiement.
- **Formats de sortie standardisés** : les résultats sont exportables en JSON, CSV ou HTML, avec une intégration native vers des plateformes de gestion des findings comme DefectDojo.

Prowler constitue ainsi le composant d'audit de posture Shift-Right, complémentaire aux scanners IaC Shift-Left : là où Checkov prévient avant déploiement, Prowler vérifie et mesure après déploiement.

2.5.4.2 Remédiation pilotée par politique

Au-delà de l'audit de posture, certains outils permettent de déclencher des **actions correctives automatisées** lorsqu'une politique est violée. *Cloud Custodian* appartient à cette catégorie et se distingue par une approche déclarative orientée ressources cloud. Sa documentation officielle définit une politique comme un objet YAML composé d'un type de ressource, d'un ensemble de filtres, d'actions et d'un mode d'exécution [21]. Cette structuration simple mais expressive permet de couvrir un large spectre de cas d'usage de gouvernance cloud :

- **Actions graduées** : Cloud Custodian supporte des niveaux de remédiation progressifs — notification (*notify*), marquage (*tag*), isolation réseau (*isolate*) et correction contrôlée (*correct*) — permettant d'adapter la réponse à la criticité du constat sans provoquer d'interruptions de service non souhaitées.
- **Modes d'exécution** : les politiques peuvent être déclenchées en mode périodique (exécution planifiée), en mode événementiel (réaction en temps quasi-réel à des événements AWS EventBridge ou Azure Event Grid) ou en mode pull sur demande.
- **Couverture multi-cloud** : Cloud Custodian prend en charge AWS, Azure et GCP avec un schéma de politique unifié, ce qui facilite la gouvernance dans des environnements multi-cloud ou hybrides.
- **Absence d'agent** : la solution fonctionne sans infrastructure dédiée permanente, réduisant la surface d'attaque et la complexité opérationnelle.

Cloud Custodian assure la remédiation automatisée des ressources cloud via des politiques déclaratives, en se basant sur leur état et événements. Dans une architecture DevSecOps, il joue le rôle de **PEP en Shift-Right** (actions correctives), complémentaire à OPA qui assure le PDP en amont.

2.5.5 Plateformes transverses de gestion des findings

Les plateformes de gestion des findings consolident les résultats de scanners hétérogènes en une couche de gouvernance unifiée. *DefectDojo* agrège les sorties de plus de 200 outils (Gitleaks, Checkov, Trivy, Prowler...), déduplique les constats entre scans répétés ou outils distincts, et assure leur traçabilité du cycle de vie (détection → correction). Il fournit des métriques de pilotage (MTTR, dette technique, tendances) [22, 23]. Contrairement aux scanners ou moteurs de politique, ces plateformes constituent une brique de gouvernance transverse essentielle à l'auditabilité et à la priorisation.

2.5.6 Synthèse comparative

Le tableau 2.4 synthétise la complémentarité fonctionnelle des catégories d'outils DevSecOps étudiées, positionnant chacune dans l'architecture Shift-Left/Right de CloudSentinel.

Table 2.4: Comparaison synthétique des principales catégories d'outils DevSecOps

Catégorie	**Exemples**	**Fonction principale**	**Critères de comparaison**	**Positionnement**
Détection de secrets	Gitleaks, TruffleHog, GitGuardian	Identifier les secrets exposés dans le code et l'historique Git	Rapidité, précision, intégration CI/CD, faux positifs	Shift-Left
Analyse IaC	Checkov, Terrascan, tfsec	Détecter les erreurs de configuration avant déploiement	Couverture technologique, expressivité des règles, rapports	Shift-Left
Analyse vulnérabilités	Trivy, Grype, Snyk	Identifier CVE et mauvaises configurations sur artefacts et images	Couverture cibles, formats de sortie, priorisation, intégration	Shift-Left
Policy-as-Code	OPA, Conftest, Sentinel, Kyverno	Évaluer des politiques et produire des décisions de gouvernance	Généralité, langage, domaine d'intégration, extensibilité	Transverse
Audit de posture	Prowler	Évaluer la conformité et posture des environnements cloud runtime	Couverture multi-cloud, référentiels, fréquence d'évaluation	Shift-Right
Remédiation pilotée	Cloud Custodian	Appliquer des actions correctives sur ressources cloud	Expressivité YAML, richesse des actions, intégration événements	Shift-Right
Gestion des findings	DefectDojo	Centraliser, dédupliquer, prioriser et suivre les constats	Déduplication, reporting, intégrations, suivi cycle de vie	Transverse

L'enjeu dans l'état de l'art n'est pas uniquement d'identifier les outils existants, mais de comprendre que leur valeur réelle émerge de leur **orchestration cohérente** au sein d'une architecture intégrée. Cette orchestration constitue l'objet de la section suivante.

2.6 Architecture d'orchestration des contrôles de sécurité

L'adoption d'outils spécialisés ne suffit pas, à elle seule, à garantir une gouvernance cohérente. Dans les environnements cloud-native, la question centrale n'est plus celle du choix des outils, mais celle de leur **orchestration** au sein d'une architecture capable de transformer des constats techniques hétérogènes en décisions cohérentes, traçables et automatisables. Cette orchestration repose sur deux mécanismes complémentaires : la normalisation des résultats issus de sources multiples, et leur intégration dans une chaîne de traitement structurée.

2.6.1 La crise de la normalisation et les contrats de données

Le point de défaillance majeur des plateformes DevSecOps actuelles réside dans l'absence de standardisation des flux de données entre les scanners et le moteur de politique. Chaque outil produit sa propre structure (Gitleaks en JSON, Checkov en SARIF, Trivy avec un schéma propriétaire). Alimenter directement OPA avec ces formats bruts rend l'architecture fragile : une simple mise à jour d'un scanner peut rompre les politiques en aval. L'ingénierie DevSecOps moderne impose donc une architecture orientée données, fondée sur les principes suivants :

- **Canonical Data Model (Modèle Canonique)** : les findings bruts doivent être transformés vers un format pivot unique, masquant l'hétérogénéité des sources. Le format SARIF tente de répondre à ce besoin pour l'analyse statique, mais échoue souvent à modéliser la complexité des configurations cloud (CSPM) et des métadonnées de contexte DevOps (branche, commit, pipeline).

- **Validation par Schéma (JSON Schema)** : le modèle canonique doit être formellement défini par un schéma versionné. Tout résultat de scan ne respectant pas ce schéma doit être rejeté avant d'atteindre le PDP. Cette validation constitue un garde-fou essentiel contre les régressions.

- **Compatibilité ascendante (Backward Compatibility)** : lorsqu'un scanner (ex : Checkov) met à jour sa structure JSON dans une nouvelle version, cela ne doit pas faire échouer les politiques OPA en aval. La couche de normalisation agit comme un **contrat d'interface** (anti-corruption layer) protégeant la logique de gouvernance des instabilités des outils tiers.

Sans la gouvernance de ces **contrats de données**, le Policy-as-Code devient impossible à maintenir : les développeurs de sécurité passent leur temps à réécrire les règles (Rego) pour s'adapter aux changements de formats des scanners, au lieu de se concentrer sur la logique métier de sécurité. Au-delà de la conversion syntaxique, cette couche de normalisation permet également : d'aligner les échelles de sévérité (CVSS, criticité métier), de dédupliquer des findings issus de sources différentes (même vulnérabilité détectée par Trivy et Snyk), d'enrichir les résultats avec des métadonnées de contexte (criticité du dépôt, environnement de déploiement), et de rendre les entrées exploitables par un moteur de décision unique.

2.6.2 Chaîne de traitement d'un pipeline DevSecOps

L'orchestration des contrôles peut être décrite comme une chaîne de traitement structurée en cinq étapes, représentée par la Figure 2.5 :

1. **Collecte des constats** : résultats produits par les scanners spécialisés (secrets, IaC, vulnérabilités, conformité), spécifiques à chaque outil à ce stade.
2. **Transformation et enrichissement** : les findings bruts sont normalisés vers le modèle canonique, contextualisés (ajout de métadonnées pipeline, environnement, branche) et consolidés dans un format commun.

3. **Décision de sécurité** : un moteur PaC (PDP) évalue les résultats consolidés à l'aide de règles formelles et produit un verdict structuré (*allow*, *deny*, *advisory*) accompagné de motifs et de métriques.
4. **Enforcement** : le PEP applique la décision : blocage du pipeline, rejet d'un artefact, émission d'une alerte ou autorisation conditionnelle avec remédiation différée.
5. **Traçabilité et retour d'information** : résultats, décisions, exceptions et actions engagées sont archivés dans une plateforme de gestion des findings (DefectDojo) pour le suivi, l'audit, l'analyse des tendances et l'amélioration continue des politiques.

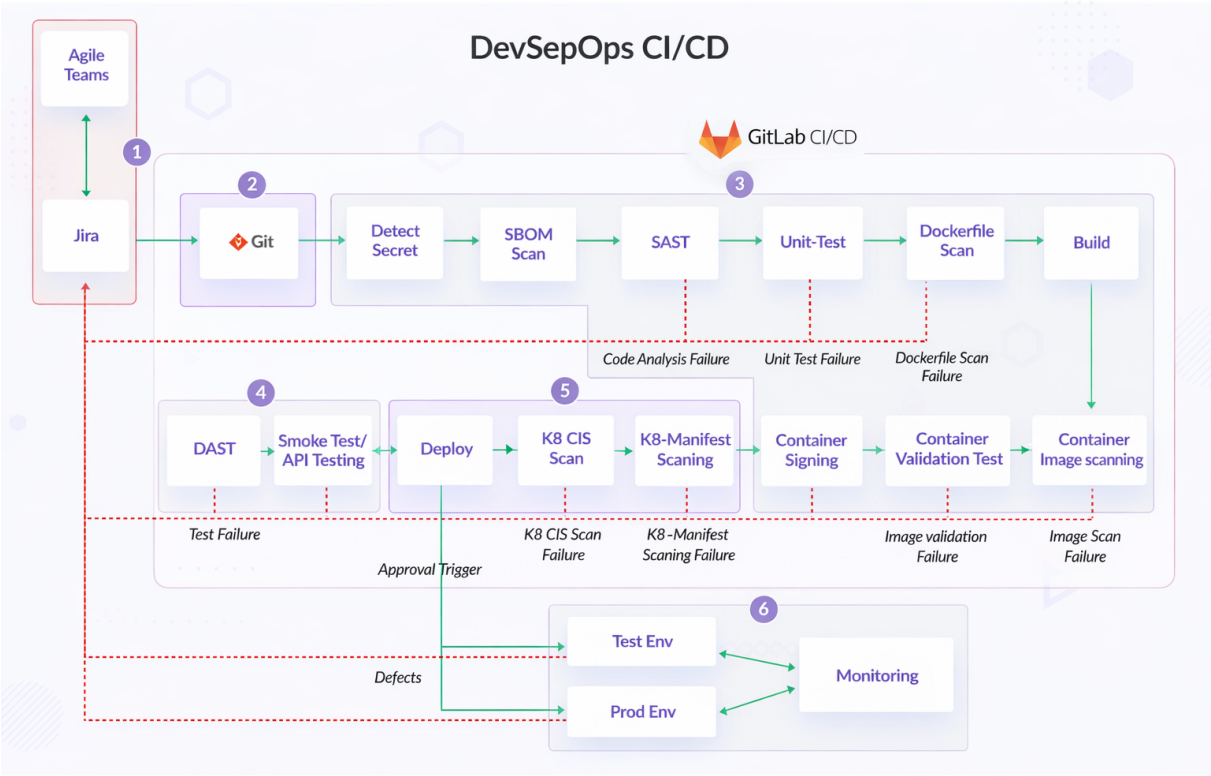

Figure 2.5: Chaîne de traitement DevSecOps : de la détection à la décision

2.6.3 Enjeux de passage à l'échelle (Scalabilité et Performance)

L'orchestration d'une chaîne DevSecOps dans une entreprise de grande taille (des centaines de dépôts, plusieurs équipes de développement, architectures microservices) ajoute des contraintes d'infrastructure strictes :

- **Performance du Pipeline (MTTR)** : si l'orchestration des scans et l'évaluation des politiques rallongent le pipeline CI/CD de plusieurs minutes, le taux de rejet par les développeurs augmente drastiquement. L'architecture d'orchestration doit privilégier les exécutions parallèles et l'évaluation asynchrone des règles non bloquantes.

- **Coût et redondance des scans** : exécuter l'ensemble des scanners sur chaque *commit* est intensivement coûteux en ressources (compute). L'orchestrateur doit être capable d'exécuter des analyses différentielles (ne scanner que les fichiers modifiés) ou de ne déclencher que les sondes pertinentes en fonction du type de changement (IaC vs code applicatif).

- **Bruit et déduplication multi-tenant** : l'agrégation de milliers de vulnérabilités génère de l'alerting asymétrique (fatigue d'alerte). La plateforme centrale (DefectDojo) doit non seulement centraliser, mais aussi corréler dynamiquement l'information issue des environnements Shift-Left et Shift-Right pour éviter la duplication des findings sur une même ressource.

Cette boucle de retour constitue un élément fondamental de la maturité DevSecOps : elle relie la détection initiale à la correction effective et au pilotage de la dette de sécurité.

2.7 Synthèse critique : lacunes dictant l'architecture cible

L'analyse de l'état de l'art montre que les outils DevSecOps existants, utilisés de façon isolée dans les pipelines CI, ne passent pas à l'échelle. Quatre limites structurent la conception d'une véritable plateforme unifiée :

1. Couplage Détection/Décision : Les règles sont codées dans les pipelines, rendant toute modification lourde et non auditable. L'architecture cible exige une **séparation stricte via PDP/PEP**.

2. Formats et API hétérogènes : Les sorties des scanners varient (JSON, SARIF, CSV), ce qui casse les politiques. Il faut une **couche de normalisation canonique** validée par un schéma stable.

3. Décision locale et simpliste : Les PDP locaux ignorent le contexte global. L'architecture doit permettre **composition de politiques et injection de métadonnées contextuelles**.

4. Rupture Shift-Left / Shift-Right : Les détections pré-prod et prod ne sont pas corrélées. Il faut un **modèle orienté événements** pour une vision continue de la dette de sécurité.

Conclusion : L'enjeu n'est plus la détection, mais la conception d'une **architecture découplée, centralisée et pilotée par les événements**, garantissant traçabilité et auditabilité. Ces principes dictent les fondements de la plateforme CloudSentinel présentée dans le chapitre suivant.

2.8 Conclusion

Ce chapitre a relié les concepts théoriques de sécurité Cloud à l'ingénierie DevSecOps. L'analyse de l'écosystème outillé (Gitleaks, Checkov, Trivy, OPA, Prowler) a montré que si les scanners sont matures, leur intégration reste fragile. Les principaux freins identifiés sont le couplage Détection/Décision, l'absence de schémas normalisés, l'isolement des décisions et la rupture Shift-Left / Shift-Right.

Le chapitre suivant détaillera la **conception de la plateforme CloudSentinel**, qui répond à ces limites grâce à un modèle événementiel, une normalisation par contrats de données et une gouvernance centralisée du cycle de vie des ressources cloud.

GENERAL CONCLUSION

APPENDICES

A.1 APPENDIX 1 TITLE

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi. Proin porttitor, orci nec nonummy molestie, enim est eleifend mi, non fermentum diam nisl sit amet erat. Duis semper. Duis arcu massa, scelerisque vitae, consequat in, pretium a, enim.

A.2 APPENDIX 2 TITLE

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi. Proin porttitor, orci nec nonummy molestie, enim est eleifend mi, non fermentum diam nisl sit amet erat. Duis semper. Duis arcu massa, scelerisque vitae, consequat in, pretium a, enim. Pellentesque congue. Ut in risus volutpat libero pharetra tempor.

PROJECT TITLE

First name Last name

Résumé:

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi. Proin porttitor, orci nec nonummy molestie, enim est eleifend mi, non fermentum diam nisl sit amet erat. Duis semper. Duis arcu massa, scelerisque vitae, consequat in, pretium a, enim. Pellentesque congue.

Mots clés: Lorem, ipsum, dolor, sit amet, consectetur, adipiscing elit.

Abstract :

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi. Proin porttitor, orci nec nonummy molestie, enim est eleifend mi, non fermentum diam nisl sit amet erat. Duis semper. Duis arcu massa, scelerisque vitae, consequat in, pretium a, enim. Pellentesque congue.

Key-words: Lorem, ipsum, dolor, sit amet, consectetur, adipiscing elit.

BIBLIOGRAPHY

[1] **DORA**. Accelerate State of DevOps 2025 Report**[online]**. Available on: https://dora.dev/dora-report-2025/

[2] **Kief Morris**. Infrastructure as Code: Dynamic Systems for the Cloud Age**[O'Reilly Media]**. Available on: https://www.oreilly.com/library/view/infrastructure-as-code/9781098114664/

[3] **GitGuardian**. The State of Secrets Sprawl 2024**[online]**. Available on: https://www.gitguardian.com/state-of-secrets-sprawl-report-2025

[4] **SentinelOne**. 50+ Cloud Security Statistics in 2026**[online]**. Available on: https://www.sentinelone.com/cybersecurity-101/cloud-security/cloud-security-statistics/

[5] **Amazon Web Services (AWS)**. Shared Responsibility Model**[online]**. Available on: https://aws.amazon.com/compliance/shared-responsibility-model/

[6] **Red Hat**. Le modèle Zero Trust, qu'est-ce que c'est ?. 2024. Available on: https://www.redhat.com/fr/topics/security/what-is-zero-trust

[7] **Xygeni**. Du DevOps au DevSecOps : Évolution des équipes de sécurité (Mise à jour)**[online]**. 2025. Available on: https://xygeni.io/fr/blog/from-devops-to-devsecops-evolution-of-security-teams/

[8] **Red Hat**. Les approches Shift Left et Shift Right**[online]**. 2024. Available on: https://www.redhat.com/fr/topics/devops/shift-left-vs-shift-right

[9] **Google Cloud / DORA Team**. 2023 State of DevOps Report**[online]**. DORA (DevOps Research and Assessment), 2023. Available on: https://cloud.google.com/solutions/devops/devops-four-key-metrics

[10] **What are DORA Metrics?**. Available on: https://www.planview.com/fr/resources/articles/what-are-dora-metrics

[11] **JIT.io**. DevSecOps: Delivery vs Security Metrics. Available on: https://www.jit.io/resources/devsecops/dora-metrics-delivery-vs-security

[12] **GitGuardian**. What is Policy-as-Code? An Introduction to Open Policy Agent**[online]**. 2024. Available on: https://blog.gitguardian.com/what-is-policy-as-code-an-introduction-to-open-policy-agent/

[13] **Open Policy Agent**. Open Policy Agent Documentation**[online]**. Available on: https://www.openpolicyagent.org/docs/latest/

[14] **Open Policy Agent**. OPA Documentation: Policy Decision Point**[online]**. Available on: https://www.openpolicyagent.org/docs/latest

[15] **Gitleaks**. Gitleaks: Scan Git Repositories for Secrets**[online]**. Available on: https://github.com/gitleaks/gitleaks

[16] **Checkov**. Checkov Documentation**[online]**. Available on: https://www.checkov.io/

[17] **Trivy**. Trivy: The All-in-One Security Scanner**[online]**. Available on: https://trivy.dev/

[18] **Trivy**. Trivy Documentation: Repository Scanning**[online]**. Available on: https://trivy.dev/latest/docs/target/repository/

[19] **Conftest**. Conftest Documentation**[online]**. Available on: https://www.conftest.dev/

[20] **Prowler**. Prowler Documentation**[online]**. Available on: https://docs.prowler.com/

[21] **Cloud Custodian**. Cloud Custodian Documentation: Overview**[online]**. Available on: https://cloudcustodian.io/docs/overview.html

[22] **DefectDojo**. DefectDojo Documentation**[online]**. Available on: https://docs.defectdojo.com/

[23] **DefectDojo**. DefectDojo Documentation: Deduplication and Findings Management**[online]**. Available on: https://docs.defectdojo.com/open_source/archived_docs/usage/features/